

CSC 411 ML Lecture 10

Reinforcement Learning

Bunmi Akinremi
AI Engineer

5th May 2026
SST Lab 2

Brief Overview

- ❑ Markov Decision Processes (MDPs),
- ❑ Value Function
- ❑ Policy Function.
- ❑ Q-Learning



● Recap of The ML Workflow

Problem Formulation

Gather and organize data

Preprocessing, cleaning, and visualizing

Choose a model and optimize

Training: Fit a new model on the training set.

Linear Models & Regularization

Non-Parametric Models & Ensembles

Support Vector Machines

Unsupervised Learning

Reinforcement Learning

Model Selection, Evaluation, and Learning Theory

Model Selection and Tuning

● The Three Pillars of Machine Learning

X

Supervised Learning

Learns from labeled examples

X

Unsupervised Learning

Finds structure in unlabeled data

✓

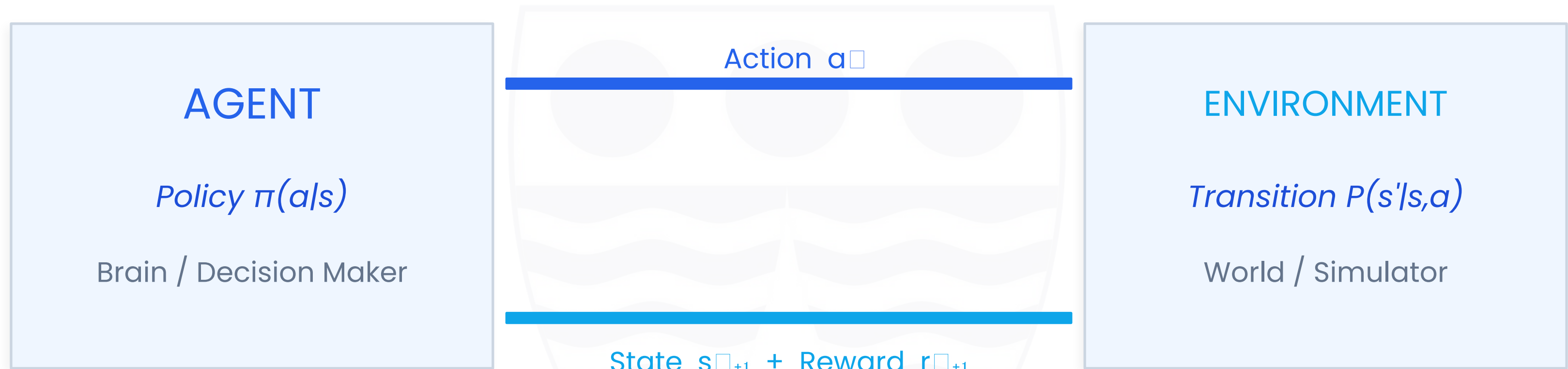
Reinforcement Learning

Learns from interaction & reward signals

● Learning by Interaction

- The core of RL is learning by interaction.
- Correct labels for the best actions are not known upfront.
- The system must discover which actions yield the most reward by trying them.

● Applications in the Real World



Goal: Find policy π^ that maximises $E[\sum \gamma^t r_t]$*

● The Agent-Environment Properties

The Reward Signal

- ❑ A scalar value $R_t \in \mathbb{R}$ received at each time step.
- ❑ The agent's sole objective is to maximize the total accumulated reward.
- ❑ Provides feedback on success or failure.

Delayed Effects

- ❑ Actions taken now may have delayed consequences.
- ❑ An action at time t may cause a reward much later.
- ❑ Challenge: The Credit Assignment Problem.

● Where is RL Applied?

Games

AlphaGo, AlphaStar, OpenAI Five beat world champions

Healthcare

Treatment planning, drug discovery, clinical trials

Robotics

Motor control, locomotion, manipulation tasks

Finance

Portfolio management, algorithmic trading, risk control

LLMs / RLHF

Fine-tuning ChatGPT, Claude via human feedback

Autonomous Vehicles

Navigation, lane-keeping, traffic optimization

● Markov Decision Processes (MDP)

RL problems are mathematically formalized as Markov Decision Processes (MDPs). It provides a theoretical framework for sequential decision-making.

MDP Formal Definition

- S** State Space
All possible situations the agent can be in
- A** Action Space
All possible actions available to the agent
- P** Transition Model
 $P(s'|s,a)$:probability of reaching s' from s via a
- R** Reward Function
 $R(s,a,s')$:immediate feedback signal
- γ** Discount Factor
 $\gamma \in [0,1]$:how much to value future rewards

The Markov Property

"The future is independent of the past given the present."

● Markov Decision Processes (MDP)

The Markov Property

$$P(s_{t+1} | s_0, a_0, \dots, s_t, a_t) = P(s_{t+1} | s_t, a_t)$$

"The future is independent of the past, given the present state."

Episodic vs. Continuing Tasks

- ❑ **Episodic Tasks:** The interaction naturally breaks into finite sequences (episodes) ending in a terminal state (e.g., winning/losing a game).
- ❑ **Continuing Tasks:** The interaction goes on continually without limit (e.g., a stock trading bot).

● Markov Decision Processes (MDP)

The Return (G_t)

- ❑ The goal is to maximize the Return G_t

The Discount Factor (γ)

- ❑ Rewards in the far future are worth less than immediate rewards.

Returns and Discounting

- *Episodic return:* $G_t = r_t + r_{t+1} + \dots + r_{t+\tau}$
- *Discounted return:* $G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}$
- $\gamma = 0 \rightarrow$ myopic (only current reward)
- $\gamma \rightarrow 1 \rightarrow$ far-sighted (values future rewards)

● Policy and Value Functions

What is a Policy (π)?

- ❑ The strategy or rules the agent follows.
- ❑ A mapping from states to actions.
- ❑ "What action should I take in this state?"

● Policy and Value Functions

Deterministic Policy

$$\pi(s) = a$$

Maps each state to a specific action. Simple but can miss optimality.

Stochastic Policy

$$\pi(a/s) = P(A_t = a / S_t = s)$$

Maps each state to a probability distribution over actions.

Optimal Policy

$$\pi^* = \operatorname{argmax}_{\pi} V\pi(s)$$

The policy that maximises expected cumulative reward for all states.

● Policy and Value Functions

Exploration vs Exploitation

The fundamental tension: *exploit known rewards (greedy) or explore unknown actions (risky but potentially better)?*

ϵ -greedy: with prob ϵ pick random action, otherwise pick $\text{argmax } Q(s,a)$.

- ❑ Exploitation: Taking the best known action to maximize reward.
- ❑ Exploration: Trying random actions to discover potentially better strategies.

The ϵ -Greedy Policy

- ❑ A simple way to balance exploration and exploitation.
- ❑ With probability $1-\epsilon$: Exploit (choose the best known action).
- ❑ With probability ϵ : Explore (choose a completely random action).

● Policy and Value Functions

State-Value Function $V_{\pi}(s)$

$$V_{\pi}(s) = E_{\pi}[G_{\infty} \mid S_0 = s]$$

Expected return starting from state s and following policy π

❑ How "good" is it to be in a particular state?

Action-Value Function $Q_{\pi}(s,a)$

$$Q_{\pi}(s,a) = E_{\pi}[G_{\infty} \mid S_0 = s, A_0 = a]$$

Expected return starting from state s , taking action a , then following π

❑ How "good" is it to take a specific action in a specific state?

● Policy and Value Functions

Key Relationship

$$V\pi(s) = \sum_a \pi(a|s) \cdot Q\pi(s,a)$$

Optimal policy: $\pi^(s) = \operatorname{argmax}_a Q^*(s,a)$*

Why They Matter

- ❑ Value functions guide policy improvement
- ❑ Most RL algorithms estimate V or Q
- ❑ Deep RL approximates them with neural nets
- ❑ They encode 'how good is it to be here?'

● The Bellman Equation

Bellman Expectation Equation

$$V\pi(s) = E[R(s,a) + \gamma V\pi(s')]$$

expanded...

$$V\pi(s) = \sum_a \pi(a|s) \sum_{s'} P(s'|s,a) [R(s,a) + \gamma V\pi(s')]$$

Current reward

Immediate feedback
from the environment

Future value

Discounted value of the
next state

Recursive nature

Value depends on
value solved by
iteration

Source: LinkedIn

● Solving MDPs with Known Dynamics

Dynamic Programming

- ❑ Used when the environment dynamics (transition probabilities and rewards) are perfectly known.
- ❑ Model-based RL.

● Solving MDPs with Known Dynamics

Policy Evaluation

- ❑ The prediction task: compute the true state-value function $v_{\pi}(s)$
- ❑ Iteratively update values using the Bellman equation until convergence.

Given policy π , compute $V_{\pi}(s)$ for all s

for each s in S :

$$V(s) \leftarrow \sum_a \pi(a|s) \sum_{s'} P(s'|s,a) [R + \gamma V(s')]$$

Policy Improvement

- ❑ The control task: make the policy better based on the newly calculated value function.
- ❑ Act greedily with respect to the new values.

Greedily update policy based on value function:

$$\pi'(s) \leftarrow \operatorname{argmax}_a Q(s,a)$$

● Model-Free Reinforcement Learning

The Unknown Environment Problem

- ❑ In the real world, we do NOT know $p(s'/s,a)$
- ❑ Example: A robot doesn't know the exact friction coefficient of the floor.
- ❑ We must learn by simulated experience.

Model-Free RL Methods

- ❑ Monte Carlo (MC) Methods
- ❑ Temporal Difference (TD) Learning

● Model-Free Reinforcement Learning

Monte Carlo (MC) Methods

- ❑ Learn directly from episodes of experience.
- ❑ Must wait until the end of the episode to calculate the true return G_t
- ❑ Update values based on average observed returns.

Temporal Difference (TD) Learning

- ❑ An extension and improvement over MC.
- ❑ Updates estimates before reaching the end of the episode.
- ❑ Learns step-by-step.

MC vs. TD Learning

- ❑ MC: Zero bias, high variance (requires full episode).
- ❑ TD: Some bias, lower variance (bootstraps). Empirically converges faster.

● Temporal Difference Learning

Core TD Update Rule: $V(S_t) \leftarrow V(S_t) + \alpha [R_{t+1} + \gamma V(S_{t+1}) - V(S_t)]$

$$R_{t+1} + \gamma V(S_{t+1})$$

TD Target

Bootstrap estimate of true value using the next step's actual reward and estimated next state value.

$$TD\ Error\ \delta_t$$

Target - $V(S_t)$

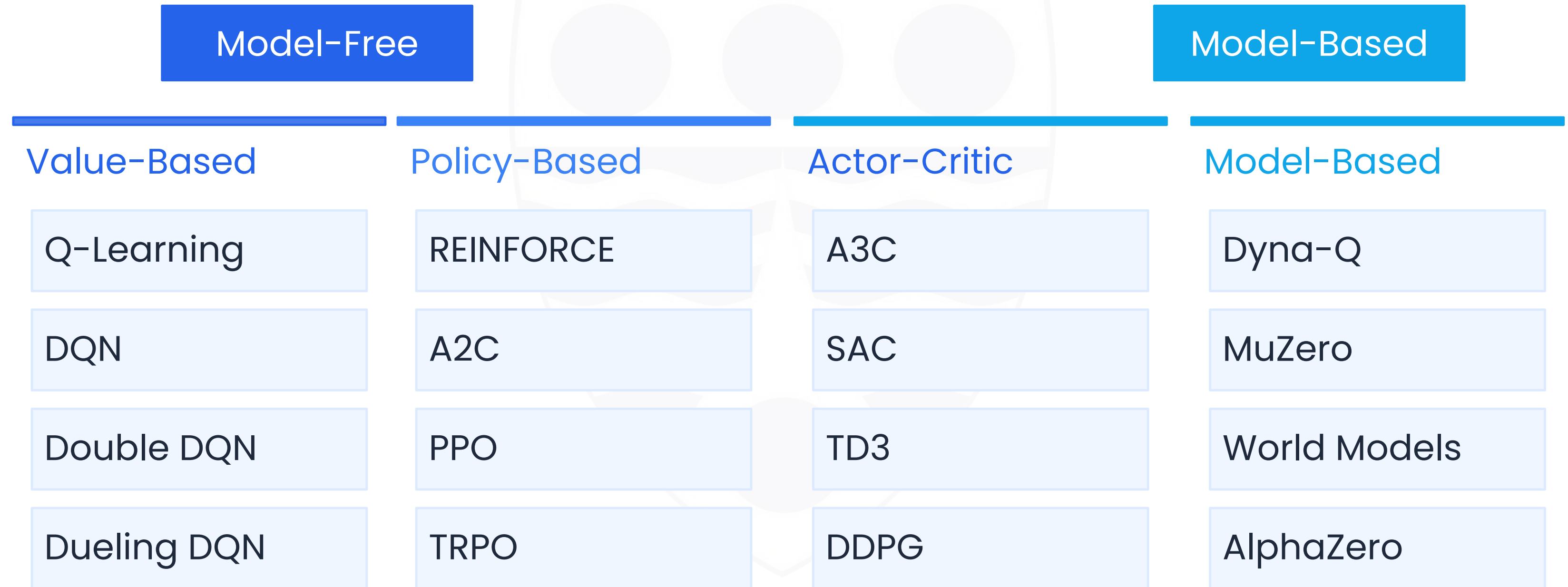
Surprise signal: how different is reality from our prediction? This drives all learning.

$$\alpha\ (alpha)$$

Learning Rate

Controls step size. Too large, unstable. Too small, slow convergence.

● RL Algorithm Landscape



● Q-Learning

SARSA: On-Policy TD Control

- ❑ Updates Q-values based on the action the agent actually took.
- ❑ Safer in practice, avoids cliff edges when exploring.

$$Q(s,a) \leftarrow Q(s,a) + \alpha [r + \gamma Q(s', a') - Q(s,a)]$$

Q-Learning: Off-Policy TD Control

- ❑ Updates Q-values based on the best possible next action, regardless of what the agent actually does.
- ❑ Can find shorter paths but may "fall off cliffs" during exploration.

● Q-Learning

The Q-Table

- ❑ A lookup table mapping every state-action pair (s,a) to a value.
- ❑ Rows = States, Columns = Actions.
- ❑ Goal: Fill the table with the optimal Q-values.

● Q-Learning

Key Properties

- ❑ Off-policy: learns optimal Q regardless of behaviour policy
- ❑ Converges to Q^* (under mild conditions)
- ❑ Simple to implement and understand

❑ The Q-Learning Update Rule

The Update Explained

$r + \gamma \max_{a'} Q(s', a')$ is the "target"

$Q(s, a)$ is the current estimate

Their difference = TD error δ

→ We nudge Q toward the target by $\alpha \cdot \delta$

● Q-Learning

Algorithm: Q-Learning (Watkins, 1989)

Initialize $Q(s,a)$ arbitrarily; $Q(\text{terminal}, \cdot) = 0$

For each episode:

Observe initial state s

For each step in episode:

Choose a from s using ϵ -greedy on Q

Take action a , observe r, s'

$Q(s,a) \leftarrow Q(s,a) + \alpha [r + \gamma \max_{a'} Q(s',a') - Q(s,a)]$

$s \leftarrow s'$

Until s is terminal

● Q-Learning

The Curse of Dimensionality in RL

- ❑ Tabular Q-learning fails if the state space is too large.
- ❑ Example: An Atari game screen has millions of pixel combinations.
- ❑ You cannot store a Q-table with millions of rows.

● Deep Q-Networks (DQN)

Neural Network Function Approximator

- ❑ Replace the Q-table with a deep CNN: $Q(s,a; \theta)$
- ❑ Input: raw pixel frames. Output: Q-values for all actions.

Experience Replay

- ❑ Store transitions (s,a,r,s') in a replay buffer D.
- ❑ Sample random mini-batches to break correlation between consecutive samples.

Target Network

- ❑ Maintain separate network θ^- for computing targets.
- ❑ Update $\theta^- \leftarrow \theta$ every C steps. Stabilises training.

● Lab 10

- Introduction to the Hugging Face Deep RL tutorial/demo workspace.
- Hands-on with stable-baselines3 and OpenAI Gym.
- Train your own agent and push it to the Hub!

Thank you

Questions?